

Quarb Implementation Guide

The Quarb Project

Version: Draft 0.1 (walking skeleton)

Contents

1	About This Guide	2
2	Architecture	3
2.1	Workspace Layout	3
2.2	The Execution Ladder	5
2.3	Current Query Subset	6
3	The Adapter Surface	9
3.1	The Live Trait	9
3.2	Worked Example: the Filesystem Adapter	12
3.3	The Planned Surface	12
4	Building and Running	13
5	Planned Integration Paths	14

Chapter 1

About This Guide

This guide describes the **Quarb engine**: the Rust implementation of the Quarb query language, its qua command-line tool, and the adapter architecture that lets one engine query many data sources. The language itself — its data model, execution model, and syntax — is defined in the *Quarb Language Specification* (the lang sibling project); this guide covers only the implementation.

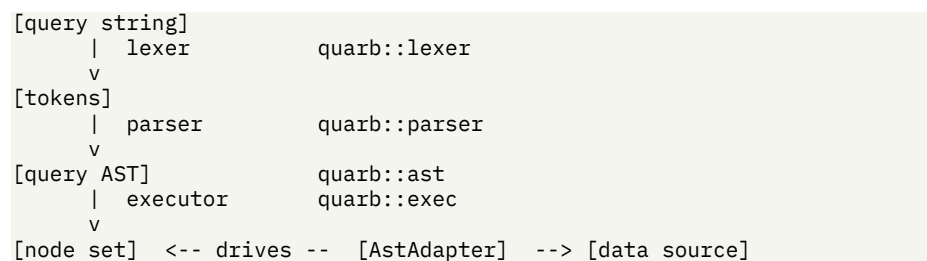
The guide tracks the code. The engine is being built as a series of thin, working slices rather than from a finished design. This document tracks that process: constructs the engine *already implements* are marked **[live]** and documented against the real code; constructs that are specified but *not yet built* are marked **[planned]** and sketch the intended surface. As each slice lands, its **[planned]** entries become **[live]**. The current milestone is a *walking skeleton*: rooted tree navigation over a filesystem.

Central concern: the adapter surface. The one design question this guide most needs to get right is the *adapter surface* — the trait a data source implements to be queried. That surface is deliberately being grown against a concrete first adapter (the filesystem) so that its shape is driven by real navigation problems rather than speculation. Chapter 3 is its home.

Chapter 2

Architecture

The engine follows a conventional compiler-style pipeline. A query string is lexed and parsed into a small AST, which the executor evaluates by driving an *adapter* over the target data:



The executor never touches the data source directly. It knows only the `AstAdapter` trait (Chapter 3), so the same query runs unchanged over any adapter.

2.1 Workspace Layout

The engine is a Cargo workspace mirroring the intended separation of concerns — the core engine, one crate per adapter, and the CLI:

Crate	Responsibility
quarb	The core engine: lexer, parser, ast, exec, and the adapter module defining AstAdapter. No I/O.
quarb-fs quarb-imap	The filesystem adapter. Depends on quarb. Remote IMAP mailboxes, queried in place: the maildir mapping over the wire (folders, messages by UID, headers as properties, :::epoch), transported by curl over subprocess (it speaks IMAP natively) — zero new dependencies. Lazy per message: one LIST, one UID SEARCH per folder, two fetches per touched message. The thread fabric is deliberately absent (indexing it means fetching every header — that is a sync; use mbsync + quarb-maildir). Target: imap[s]://HOST[:PORT]/FOLDER; credentials via QUARB_IMAP_USER/PASS or ~/.netrc. Read-only. Tested against a bundled deterministic mini IMAP server.
quarb-json	The JSON adapter (keys and indices as names, value types as traits). Depends on quarb and serde_json.
quarb-gsheet	Google Sheets on the quarb-xlsx mapping (sheets as tables, first-row headers, typed values via UNFORMATTED_VALUE), fetched from the Sheets API. Auth: the API rejects gcloud's cloud-platform tokens, so the friction-free path is an API key (?key= / QUARB_GSHEET_KEY; enough for link-readable sheets), with a properly-scoped bearer for private ones. Target: gsheets://SPREADSHEET_ID.
quarb-html	The HTML adapter (tags as names, block/inline as traits, attributes as properties, anchor fragments resolve). Depends on quarb and scraper.
quarb-pdf	PDF internals as the object graph: indirect objects are nodes named by their declared /Type, scalar dictionary entries are properties, references are edges labeled by their key (->Kids, <-Parent; array references hoist to the array's owner), with the reverse index built at load. Depends on quarb and lopdf.
quarb-text	The shared text-level model — the relational pattern for documents: the per-format crates lower their format into one block-event vocabulary (the atrep shape: kind, taxis, lemma, body, hypograph), and this crate alone implements the adapter. Sections with explicit title (::lemma) + body are derived from the flat heading stream by the outline rule; paragraphs, blockquotes (attribution as ::hypograph), lists, and

Adapters are separate crates by design: `quarb-fs`, `quarb-json`, `quarb-html`, and `quarb-xml` share nothing but the `AstAdapter` trait, and a consumer pulls in only the adapters it needs. Each new adapter has confirmed the surface without changing it — the filesystem, a JSON tree, an HTML DOM, and an XML document are genuinely the same arbor to the engine.

2.2 The Execution Ladder

Database adapters evaluate on a three-rung ladder; every rung is *output-identical* to the one below it (differential tests hold this), so the rungs trade only latency and transfer:

1. **Full pushdown** (`quarb_sql::pushdown`, the exporter's strict mode): the whole query compiles to SQL when every construct is in the verified-safe set — WHERE-filtered selections, whole-table aggregates, witness joins. Strictness is the point: anything whose SQL semantics are not provably identical (LIKE case folding, truthiness, GROUP BY/DISTINCT order, collations, unordered LIMIT) refuses. Row order is part of the contract — the plan names the result table, and each driver appends ORDER BY its key from the catalog. Measured: COUNT over 500k PostgreSQL rows, 2.57 s scanned vs. 0.018 s pushed.
2. **Partial pushdown** (`quarb_sql::partial_pushdown`): the leading run of strictly-translatable row predicates becomes the table's fetch filter (each driver's lazy fetcher accepts a per-table WHERE), and the *original* query runs unchanged over the reduced set — the engine re-applies the pushed predicates, a no-op on pre-filtered rows, so there is no query rewriting. Gates: leading run only (a positional predicate before an expression one sees unfiltered rows), a single `/table/*` branch with no correlations, the table reached exactly once, no crosslink/resolution axes, no `:::/:::` metadata (each would observe the filtering). Since the JSON-column round (2026-07-31), refused predicates in the leading run degrade to *superset prefilters* instead of aborting the plan: fixed-path JSON comparisons emit per-dialect forms proven never to drop a matching row (SQLite typed extract with a `json_type='text'` escape hatch, PostgreSQL `jsonb_typeof` CASE with `::float8`; existence and `!=` floor at `json_type IS NOT NULL`) — extras are re-checked by the re-run, and a server-rejected prefilter surfaces through an eager prefetch and falls back to the scan rather than answering empty. Full JSON string-equality pushdown carries the dialect's validity guard (`json_valid` and `kin`) and refuses numeric-looking literals (Quarb's `value_eq` coerces those numerically; no extractor matches that). Measured: unpushable grouping over a 1%-selective filter, 2.31 s vs. 0.081 s; JSON numeric compare over 200k rows, 1.46 s scanned vs. 0.25 s pre-filtered.

3. **Lazy scan:** catalog-eager, rows-lazy per table (Section on `quarb-relational`); correct for everything, always the fallback.

`qua --explain` prints the decision chain; `qua --no-pushdown` pins the scan rung; `qua --save FILE [--as NAME]` materializes any result as a first-class input (.db table / .json array) so expensive reductions run once and iteration proceeds locally. For metered engines (a future BigQuery adapter bills by bytes scanned), the ladder's transfer reductions are cost reductions.

2.3 Current Query Subset

The engine implements essentially the whole language. A query is a root-anchored sequence of hops, each an axis followed by a name:

Form	Meaning
/name, //name	[live] child / de- scen- dant named
\name, \\name	named name [live] par- ent / an- ces- tor named
>name, <name	named name [live] next / pre- vi- ous sib- ling
>>name, <<name, >>?, >>!, <<?, <<!	[live] sib- ling reach: all fol- low- ing / pre- ced- ing sib- lings at any dis- tance, with prox- i- mal / dis- tal quan- ti- fiers (matcher

The pipeline runs on *capsae* — the register system’s unit, a node paired with a register (its breadcrumbs) and a current topic. A per-capsa `|` transforms each topic; `@|` reduces the whole context; `| .name` pushes the topic onto the register and `| $.name` recalls it; a subcontext `| .(expr)` evaluates `expr` from each node, reduces it to a scalar, and pushes it — grouped aggregation without leaving the query. A push quantified `{N}` repeats over the whole context, each round seeing the last (the iteration loop); `X: .r` reads the register of whichever capsae stands at the node `X` reaches, from the same snapshot; and every value remembers where it was read, so `:::` on a value answers its provenance the way `:::` on a node does.

Any other operator is lexed and rejected with a `not yet supported` error naming the offending construct, rather than silently mis-parsed. Everything in Section 3.3 falls into this category today.

Chapter 3

The Adapter Surface

An *adapter* maps a data source onto the Quarb arbor model — a tree backbone whose edges carry *names*, enriched with non-hierarchical crosslinks. The engine drives all navigation through the `AstAdapter` trait, so this trait is the contract between the language and the world.

3.1 The Live Trait

The methods the skeleton actually uses. Signatures are the real ones from `quarb/src/adapter.rs`:

```
/// An opaque handle to a node, minted and interpreted by the
/// adapter that produced it.
pub struct NodeId(pub u64);

pub trait AstAdapter {
    /// The root node -- the initial navigation context.
    fn root(&self) -> NodeId;

    /// The tree children of `node`, in document order.
    fn children(&self, node: NodeId) -> Vec<NodeId>;

    /// The name of `node` -- the label of its incoming tree edge.
    /// `None` when the adapter leaves a node unnamed (e.g. the
    /// filesystem root).
    fn name(&self, node: NodeId) -> Option<String>;

    /// The parent of `node`, or `None` for the root.
    fn parent(&self, _node: NodeId) -> Option<NodeId> { None }

    // --- projection surface (defaulted; implement what fits) ---

    /// Adapter-defined classifications, for `<trait>` filters.
    fn traits(&self, _node: NodeId) -> Vec<String> { Vec::new() }

    /// A named property `::prop`.
}
```

```

fn property(&self, _node: NodeId, _name: &str) -> Option<Value> {
  None }

/// The default projection, bare `::` (filesystem: file content).
fn default_value(&self, _node: NodeId) -> Option<Value> { None }

/// Adapter metadata `:::key` (filesystem: size, modified, ...).
fn metadata(&self, _node: NodeId, _key: &str) -> Option<Value> {
  None }

/// Outgoing / incoming crosslinks as (label, node) pairs, for
/// `->` / `<-` (filesystem: symlinks).
fn links(&self, _node: NodeId) -> Vec<(String, NodeId)> { Vec::new
() }
fn backlinks(&self, _node: NodeId) -> Vec<(String, NodeId)> { Vec::
new() }

// --- the reference surface (defaulted) ---

/// Resolve a cross-reference: `::prop-->hint` maps a property
/// value to its in-document target (html: a `#fragment` href
/// to the element with that id).
fn resolve(&self, _n: NodeId, _prop: &str, _hint: Option<&str>)
  -> Option<NodeId> { None }

/// The external reference a property holds -- an
/// adapter-defined identifier of a document outside the arbor
/// (html: the absolute URL, relative hrefs joined against the
/// document's declared URL). On an in-document miss the engine
/// consults the mount layer's reference targets (identifier ->
/// mounted root) and lands, or records the identifier among
/// the run's unresolved external references for the host's
/// acquisition loop. The engine performs no IO, ever.
fn external_ref(&self, _n: NodeId, _prop: &str, _hint: Option<&str
>)
  -> Option<String> { None }

/// The element `fragment` names inside `node`'s document --
/// the landing rung of a fragment-carrying external reference.
fn resolve_fragment(&self, _n: NodeId, _frag: &str)
  -> Option<NodeId> { None }

/// The property carrying the node's own reference, for the
/// bare arrow `//a-->` (html: an anchor's href); and the
/// relation an edge carries (html: `rel`), the label falling
/// back to the property name.
fn ref_property(&self, _n: NodeId) -> Option<String> { None }
fn ref_label(&self, _n: NodeId, _prop: &str) -> Option<String> {
  None }

// --- the index hooks (defaulted; an indexed store answers) ---

/// The document an external identifier names (a URL -> its
/// page), the arrow's landing rung after the mount layer.
fn document_by_ref(&self, _id: &str) -> Option<NodeId> { None }
/// `<-` from an index instead of a whole-arbor scan.
fn reverse_resolve(&self, _n: NodeId, _prop: Option<&str>,

```

```

    _hint: Option<&str>) -> Option<Vec<NodeId>> { None }
    /// `//name` below `node` from an index, with each hit's depth;
    /// `None` means walk. The answer must equal the walk's.
    fn descendants_named(&self, _n: NodeId, _name: &str)
        -> Option<Vec<(NodeId, usize)>> { None }
    /// A batching hint before a link sweep over `nodes`.
    fn prefetch_links(&self, _nodes: &[NodeId], _dir: LinkDir) {}

    // --- identity and provenance (defaulted) ---

    /// Whether `node` is a node of this arbor -- the guard of the
    /// node-id anchor `(!expr)`. Default: probe parent and name
    /// under catch_unwind.
    fn has_node(&self, node: NodeId) -> bool { ... }
    /// What the source recorded about `node`: source, path within
    /// it, instant (and which rung it came from), dpid; a list when
    /// several sources stand behind one node (mounts, grafts,
    /// models layer entry by entry).
    fn provenance(&self, _n: NodeId) -> Provenance { ... }
    fn provenance_list(&self, n: NodeId) -> ProvenanceList { ... }
}

```

The navigation methods are required; the projection methods carry defaults, so an adapter implements only what its domain offers. Core metadata (: : :key — index, depth, counts, is-leaf, ...) is computed by the engine from the navigation surface, so no adapter implements it.

Three observations that the filesystem work has already forced:

- **Names live on edges, not nodes.** `name` returns the label of a node's *incoming* edge — its role in its parent — exactly as the specification's edge-name model requires. For the filesystem this is simply the file-name.
- **NodeId is opaque and adapter-private.** The engine never inspects the u64; it only passes ids back to the adapter. Each adapter chooses its own identity scheme (the filesystem adapter uses an interning arena index).
- **New trait methods need their forwarders.** The run path wraps every adapter in decorators (`WithNow`, `AllowShell`), the session in `Dyn`, mounts in `MountAdapter/Shared`, models in `ModelAdapter` — a defaulted method silently answers `None` through any wrapper that does not forward it. When adding to the trait, sweep every delegating impl. The case in point (2026-09-07): `Dyn` had never forwarded `answers_to`, so a web level behind the session's boxed door answered `//page` with nothing — its rows answer to their kind through that method. It forwards the whole surface now.
- **The root may be unnamed.** `name` returns `Option` precisely so an adapter can leave the root without a name, as the filesystem root / does.

3.2 Worked Example: the Filesystem Adapter

quarb-fs implements the live trait over a directory tree. It materializes the tree once, at construction, into arrays indexed by NodeId (path, children, parent); the trait methods are then trivial lookups:

```
impl AstAdapter for FsAdapter {
  fn root(&self) -> NodeId { self.root }
  fn children(&self, node: NodeId) -> Vec<NodeId> { self.children[
    node].clone() }
  fn name(&self, node: NodeId) -> Option<String> {
    if node == self.root { return None; } // "/" is unnamed
    self.paths[node].file_name().map(|n| n.to_string_lossy().into())
  }
  fn parent(&self, node: NodeId) -> Option<NodeId> { self.parents[
    node] }
}
```

Ignore rules [live]. The materializing walk uses `ignore::WalkBuilder` — the same machinery ripgrep uses — so by default the adapter respects `.gitignore/.ignore` files and skips hidden entries. This came directly from running the skeleton: `qua '/*.*.rs'` in the engine's own tree first returned a build artifact under `target/`, because nothing yet excluded it. The adapter now honors ignore rules by default, and `qua` exposes `--hidden` and `--no-ignore` to opt back in. The walk is currently eager (a targeted query still walks the whole tree); a lazy traversal with cascading ignore rules is a planned optimization. This is the pattern the guide follows: a concrete problem, surfaced by running the code, drives the next slice.

3.3 The Planned Surface

The rest of the arbor model, specified in the language but not yet built. Each becomes a live trait method (or a related capability) as its engine slice lands. The intended shapes:

Capability	Query	Adapter surface (sketch)
Pattern search	=>	<code>search(pattern) -> Vec<NodeId>.</code>

These signatures are provisional. They are recorded here so the direction is legible, not to freeze a contract — the filesystem adapter (and the JSON adapter after it) will reshape them as they are implemented.

Chapter 4

Building and Running

```
# from the engine workspace root
cargo build           # build the whole workspace
cargo test            # unit + integration + doc tests

# query a directory tree
cargo run -p qua -- '//*.rs' src/      # all Rust files under src/
cargo run -p qua -- '/README.md'       # a single top-level file

# query a document
cargo run -p qua -- '/users/0/name::' data.json
cargo run -p qua -- '//book[::pages > 200]:::id' catalog.xml
cargo run -p qua -- '//a::href' page.html
```

qua <QUERY> [PATH] runs QUERY against the directory PATH (default: the current directory) and prints each matching path on its own line, so it composes in shell pipelines alongside sort, uniq, and friends.

Chapter 5

Planned Integration Paths

Beyond the direct `AstAdapter` implementation shown above, these integration paths are described in the specification but **[planned]** (none is implemented yet):

- **Petgraph adapter.** A ready-made adapter over a `petgraph::Graph`, for Rust callers who transform their AST into a graph once and query it at speed. See `petgraph-adapter.md` in this project for the design note.
- **Python bindings.** A `quarb` Python package exposing the engine and an `AstAdapter` base class.
- **C FFI.** A C ABI for embedding the engine in other languages.

Each will be added once the core language surface is substantial enough to be worth binding.